

CSE 220 -- DFT / FFT Practice Question Bank: Big-Integer Track

Companion practice set for the bigmul.py (Task A) spectral multiplication lab
5 independent problems, each sized for a 40-45 minute closed-lab test

How to use this document

Each problem below is written in the same format as bigmul.py's own specification: a short mathematical model, 2-3 TODO-sized functions, a fixed mark allocation (10 marks total), a run command, and a verification rule against an independent oracle. Every problem reuses the infrastructure you already built -- **transforms.py** (DFTAnalyzer / FFTTransformer / ArbitraryLengthFFT), and the exact limb/carry conventions from **bigmul.py** (to_limbs / from_limbs / multiply_transform / multiply). Do not modify transforms.py, io_utils.py, or bench_utils.py; you may freely copy logic out of your own completed bigmul.py into the matching TODOs below.

The matching solving scaffold is **practice_bigmul_template.py**, provided separately. It imports your real bigmul.py so multiply(), to_limbs(), and from_limbs() are already available to call from any problem.

As with the image-domain question bank, every problem here is intentionally **harder** than the original lab: two require designing a new numerical algorithm around your existing spectral multiply() (fast squaring, Newton-Raphson division), one strips away exact carry handling to expose circular-convolution corruption on purpose, one builds modular exponentiation on top of your transform routines, and one requires a from-scratch divide-and-conquer base-conversion routine.

Problem 1 -- Fast Squaring via a Single Spectral Transform

Marks: 10 | Suggested time: 40 minutes

Squaring a limb array A is a special case of `multiply_transform(A, A, engine)` that wastes half its work: A is transformed twice even though it is the same array both times. This problem asks you to compute A^2 with exactly ONE forward transform and one inverse transform, by squaring the spectrum itself:

$$Y = \text{IFFT}(\text{FFT}(A)^2)$$

- **`square_via_transform(limbs, engine)`** -- zero-pad limbs to $N = \text{next_power_of_two}(2 \cdot \text{len}(\text{limbs}) - 1)$ (or the exact linear length for a non-FFT engine), transform ONCE, square the spectrum elementwise (not multiply two separate transforms), inverse-transform, round to the nearest integer, and return the uncarried convolution coefficients together with N -- same contract as `multiply_transform`. [5]
- **`square_decimal(text, method)`** -- convert text to limbs with `to_limbs`, dispatch to `square_via_transform` for the chosen engine (or a schoolbook self-multiply for `method='schoolbook'`), carry with `from_limbs`, and re-attach the sign (a negative number squared is always positive -- do not just reuse the operand's sign). [3]
- **`run_square(path, method, out_dir)`** -- read ONE operand per input file (reuse `io_utils.read_operands` but only take the first line, or write a one-line reader), write `square.txt` and `report.txt` with the same fields as `bigmul.py`'s `run_single`, and verify against `int(text_a) ** 2`. [2]

Run:

```
python3 practice_bigmul_template.py p1 inputs/1.txt --engine fft --out-dir outputs/p1
```

Verification. MATCH/MISMATCH against Python's `int(text_a) ** 2`, exactly as `run_single` does in `bigmul.py`. The harness additionally times `square_via_transform` against calling your own `multiply_transform(A, A, engine)` on the same input and checks that the single-transform version is measurably faster for a large operand (≥ 20000 digits).

Trap: `engine.transform(x)**2` squares each complex spectrum value $C = a+bi$ as $C^2 = a^2 - b^2 + 2abi$ -- this is NOT the same operation as re-running the forward transform a second time and it must be done with a single elementwise NumPy squaring, not a second call to `engine.transform`.

Problem 2 -- Big-Integer Division via Newton-Raphson Reciprocal

Marks: 10 | Suggested time: 45 minutes

Division is the one basic operation `bigmul.py` never implements. This problem builds `floor(A / B)` and `A mod B` for two huge positive decimal integers using Newton's iteration for a reciprocal, so that division reduces to a handful of calls to your own `multiply()` -- no schoolbook long division and no Python big-integer division of the full operands.

Newton's iteration for $x \approx 1/B$, scaled by 10^p (p a fixed working precision in decimal digits), refines a guess x_k via

$$x_{k+1} = x_k * (2 * 10^p - B * x_k) / 10^p$$

starting from a crude estimate based on B 's digit count, and doubling the number of correct digits roughly every iteration.

- **`newton_reciprocal(divisor_text, precision_digits, method)`** -- return a decimal string representing `floor(10precision_digits / int(divisor_text))` using ONLY the iteration above (implemented with your `multiply()` for every $A*B$ -sized product and ordinary string/limb arithmetic for the additions, subtractions and the fixed power-of-ten scaling/truncation). Iterate until two successive iterates agree in their leading `precision_digits - 10` digits. [5]
- **`divmod_big(text_a, text_b, method)`** -- use `newton_reciprocal` to get an approximate quotient $q_0 = \text{floor}(A * \text{reciprocal}(B) / 10^{\text{precision_digits}})$, then correct it: while $q_0 * B$ (via `multiply()`) exceeds A , decrement q_0 ; while $(q_0+1)*B$ is still $\leq A$, increment q_0 -- at most a small constant number of corrections are ever needed if `precision_digits` was chosen with a few extra guard digits. Return `(quotient_text, remainder_text)`. [4]
- **`run_divmod(path, method, out_dir)`** -- read two operands, call `divmod_big`, write `quotient.txt` and `remainder.txt`, and verify against Python's built-in `divmod()`. [1]

Run:

```
python3 practice_bigmul_template.py p2 inputs/1.txt --engine fft --out-dir outputs/p2
```

Verification. `(quotient_text, remainder_text)` must equal `divmod(int(text_a), int(text_b))` exactly, printed as MATCH/MISMATCH; remainder must additionally satisfy $0 \leq \text{remainder} < B$ and $\text{quotient} * B + \text{remainder} == A$ when checked with the same oracle.

Trap: `precision_digits` must exceed `len(A)`'s digit count by a safety margin (a handful of guard digits is standard) or the correction loop in `divmod_big` may need many iterations instead of $O(1)$ -- that is a sign the working precision was chosen too tight, not a sign the correction loop is wrong.

Problem 3 -- Modular Exponentiation of Huge Integers by Repeated Spectral Squaring

Marks: 10 | Suggested time: 45 minutes

Compute $\text{base}^{\text{exponent}} \bmod \text{modulus}$ for a huge decimal base and exponent, where modulus fits comfortably in a machine integer (up to about 15 decimal digits). Binary (square-and-multiply) exponentiation keeps every intermediate value reduced modulo `modulus` after each spectral multiplication, so intermediate operands never blow up to the size of $\text{base}^{\text{exponent}}$ itself.

- **mod_small(text, modulus)** -- compute `int(text) % modulus` using Horner's rule digit by digit (`acc = (acc*10 + digit) % modulus` for every decimal digit of text, left to right), with NO call to Python's `int(text)` on the full string and no big-integer division. [3]
- **modpow_big(base_text, exponent_text, modulus, method)** -- standard binary exponentiation: walk the BINARY digits of `int(exponent_text)` (you may convert the exponent itself with `int()` -- only the base and the running result are treated as "big"), repeatedly squaring the running result with your `multiply()/square_via_transform` machinery and reducing with `mod_small` after every squaring and every multiply-in of the base. Return the final result as a plain Python `int` (it is at most `modulus-1`, so this is safe). [6]
- **run_modpow(path, modulus, method, out_dir)** -- read base and exponent as the two lines of the input file, call `modpow_big`, write `result.txt` and `report.txt`, and verify against Python's built-in three-argument `pow()`. [1]

Run:

```
python3 practice_bigmul_template.py p3 inputs/1.txt --modulus 998244353 \  
--engine fft --out-dir outputs/p3
```

Verification. Result must equal `pow(int(base_text), int(exponent_text), modulus)` exactly -- report MATCH/MISMATCH. The harness also checks `exponent = 0` (result must be `1 % modulus`) and `base already >= modulus` (must be reduced by `mod_small` before the first squaring).

Trap: the exponent's OWN size is not the bottleneck (it is read with an ordinary `int()` call, exactly like `NTT_MOD/NTT_ROOT` are plain Python ints elsewhere in `bigmul.py`) -- the point of this problem is keeping the huge BASE reduced after every squaring, not avoiding `int()` everywhere indiscriminately.

Problem 4 -- Circular (Wraparound) Big-Integer Multiplication: Finding Where It Breaks

Marks: 10 | Suggested time: 40 minutes

`multiply_transform` in `bigmul.py` always chooses `N` large enough for the LINEAR convolution. This problem asks you to deliberately choose an insufficient circular length `N` and measure exactly how much of the product gets corrupted -- the 1D, big-integer analogue of comparing circular vs. linear image convolution.

- **`multiply_circular(limbs_a, limbs_b, N, engine)`** -- zero-pad both limb arrays to EXACTLY length `N` (raise `ValueError` if `N < max(len(limbs_a), len(limbs_b))`); `N` need not be large enough for the true linear convolution), transform, multiply spectra, inverse-transform, round to integers -- the result represents $(A*B)$ reduced modulo $(\text{BASE}^N - 1)$ at the limb level, i.e. deliberately wrapped. [4]
- **`decode_possibly_wrong(sign, wrapped_limbs, base_digits)`** -- run the wrapped limbs through the SAME carry-propagation logic as `from_limbs` (reuse it directly) to obtain a decimal string, even though the result may be numerically wrong -- `from_limbs` itself must not be modified or special-cased. [2]
- **`first_divergence(text_a, text_b, N, method)`** -- compute the correct product via your existing `multiply()` and the deliberately-wrapped product via `multiply_circular`, and return the index (counting from the least-significant digit, 0-based) of the first decimal digit at which the two decimal strings differ, or -1 if `N` happened to be large enough that they match. [4]

Run:

```
python3 practice_bigmul_template.py p4 inputs/1.txt --N 64 --engine fft --out-dir outputs/p4
```

Verification. The harness sweeps `N` from clearly-too-small up to `next_power_of_two(linear length)` and checks that `first_divergence(...)` returns -1 only once `N` reaches the true linear-convolution length, and returns a non-decreasing, roughly `N`-proportional divergence point for smaller `N` (larger `N` corrupts fewer of the product's low-order digits).

Trap: 'wrapping at the limb level' is not the same as wrapping at the decimal-digit level -- `BASE_DIGITS=4` decimal digits live inside each limb, so an error that first appears at limb index `k` first appears at decimal digit index roughly `4*k`, not `k`. Off-by-a-factor-of-base_digits answers are the most common mistake here.

Problem 5 -- Fast Radix Conversion: Decimal to Base 65536 by Divide-and-Conquer

Marks: 10 | Suggested time: 45 minutes

Converting an n -digit decimal number to another base by repeated small division is $O(n^2)$. This problem builds an $O(n \log^2 n)$ converter instead, using divide-and-conquer: split the decimal string into a high half and a low half, convert each half recursively, and recombine as $\text{high} \cdot 10^k + \text{low}$ using your spectral `multiply()` to compute the large power-of-ten multiplication, not repeated small multiplications.

- **`multiply_base_arrays(a, b, base, engine)`** -- convolve two little-endian digit arrays that are already expressed in an arbitrary base (not necessarily 10^4) and carry-propagate in that base -- the same shape as `multiply_transform + from_limbs` in `bigmul.py`, but generalised to take `base` as a parameter instead of hardcoding `BASE`. [3]
- **`base_pow65536(exponent, base_from, engine)`** -- return the base-65536 little-endian digit array representing `base_from`^{exponent} (`base_from` will always be 10 in this problem), computed by REPEATED SQUARING using `multiply_base_arrays` -- not by naive repeated multiplication by 10, which would defeat the point of using a spectral multiply at all. [4]
- **`decimal_to_base65536(text, engine, small_cutoff=8)`** -- if `len(text) ≤ small_cutoff`, convert directly with ordinary Python arithmetic on that small chunk (this mirrors `bigmul.py`'s own `to_limbs`, which uses `int()` freely on individual 4-digit limbs); otherwise split `text` into a high part (all but the last `half*2` digits, rounded so the low part's length is even) and a low part of exactly `half` decimal digits, recursively convert both halves, and combine as `high_digits` (multiplied by `base_pow65536(half, 10, engine)` via `multiply_base_arrays`) plus `low_digits` (as a big-array addition with carries). [3]

Run:

```
python3 practice_bigmul_template.py p5 inputs/1.txt --engine fft --out-dir outputs/p5
```

Verification. Reconstruct the decimal value from your base-65536 array with an independent, separately-written Horner evaluator ($\sum d_i \cdot 65536^i$ using Python's big integers -- this reverse direction is the ONE place ordinary big-integer arithmetic is allowed, exactly as `run_single`'s oracle step is the one allowed place in `bigmul.py`) and confirm it equals `int(text)` exactly; report `MATCH/MISMATCH`.

Trap: `'high * 10k + low'` only works if `low` is zero-padded to EXACTLY `k` decimal digits before the split (leading zeros in the low half must be preserved going into the recursion) -- dropping leading zeros from the low half is the classic bug that makes this kind of divide-and-conquer conversion silently wrong only for inputs whose low half happens to start with a zero digit.

Summary Table

P1 -- Fast squaring, single transform (spectrum-squaring instead of two forward transforms)

P2 -- Newton-Raphson big division (reciprocal iteration built on multiply())

P3 -- Modular exponentiation (keeping a huge base reduced mod a small modulus)

P4 -- Circular wraparound multiplication (1D analogue of image aliasing)

P5 -- Divide-and-conquer radix conversion ($O(n \log^2 n)$ base-65536 conversion)

Reminder: in the real test you will not be told which single problem you receive until you open it, and (as in bigmul.py itself) Python's own big integers may be used ONLY to check your answer at the end -- never as part of the algorithm you are asked to implement.